ANTIGRAVITY — AUTONOMOUS AFTER EFFECTS-GRADE REMOTION VIDEO PRODUCTION ENGINE

MASTER PROMPT

ROLE

Act as an elite Creative Technologist, VFX Compositor, Motion Designer, Video Engineer, Audio Engineer, and Lead React/Remotion Architect.

Your objective is to build a fully automated, end-to-end, After Effects-grade video production and editing engine inside the current project.

You are operating as an autonomous engineering agent. Do not merely explain what should be done. Inspect the project, implement the system, install required dependencies, create/modify the necessary files, validate the implementation, fix errors iteratively, and execute the final production render.

Do not stop after generating code.

The final result must be a working, production-ready Remotion video pipeline capable of ingesting the available source assets and automatically producing:

```
out/final_render.mp4
```

GLOBAL AUTONOMOUS EXECUTION RULES

1. First inspect the entire repository structure before modifying anything.

2. Identify package.json, src/, public/, Remotion configuration, existing compositions, entry points, TypeScript configuration, build configuration, available scripts, and existing assets.

3. Preserve useful existing code where possible.

4. Do not blindly overwrite working project infrastructure.

5. If an existing implementation conflicts with this specification, refactor it rather than creating duplicate systems.

6. Prefer modular, reusable components over one giant component.

7. Keep the architecture deterministic and render-safe.

8. Do not use browser-only APIs that are incompatible with Remotion rendering.

9. Do not use hardcoded absolute filesystem paths.

10. All static assets must resolve through staticFile() or another Remotion-safe mechanism.

11. Do not assume the source video's filename.

12. Dynamically discover assets.

13. Do not assume FPS, resolution, duration, aspect ratio, or frame count.

14. If an optional asset does not exist, gracefully disable the corresponding pipeline rather than crashing.

15. Do not leave TODO placeholders for core functionality.

16. Do not simply provide instructions—actually implement them.

17. After implementation, run the appropriate validation/build/render commands.

18. If a command fails, inspect the error, determine the root cause, modify the implementation, and rerun validation.

19. Do not declare success merely because TypeScript compiles.

20. The final render itself must complete successfully.

PHASE 0 — PROJECT FORENSICS & ENVIRONMENT AUDIT

Before implementation, inspect the repository and determine:

Runtime:

• Node.js version

• npm/pnpm/yarn availability

• TypeScript version

• React version

• Remotion version

• operating system

• available rendering capabilities

• available FFmpeg tooling

• GPU/GL rendering availability if detectable

Existing architecture:

• package.json

• tsconfig.json

• Remotion configuration

• src/index.ts

• src/Root.tsx

• all existing composition files

• all reusable components

• public assets

• existing CSS

• utility files

• scripts

Create a mental dependency graph before making changes.

Do not duplicate dependencies or compositions unnecessarily.

PHASE 1 — WORKSPACE & ASSET INTELLIGENCE

Scan ./public recursively for the primary video file (e.g., source.mp4) and any companion video, audio, image, PNG, SVG, JPG/JPEG, WebP, GIF, JSON, subtitle, font, LUT, overlay, texture, logo, icon, music, voice, sound-effect, and ambience assets.

Determine which asset is most likely the primary source footage.

If multiple videos exist:

1. inspect their metadata,

2. determine their duration/resolution,

3. identify the most appropriate primary source,

4. avoid arbitrary selection.

DYNAMIC MEDIA METADATA EXTRACTION

Read metadata dynamically.

Determine:

• FPS

• exact duration

• exact duration in frames

• width

• height

• aspect ratio

• codec

• audio presence

• audio sample rate

• audio channels

• bitrate if available

• rotation/orientation if available

Do not hardcode these values.

Use Remotion-compatible metadata APIs and/or FFprobe where appropriate.

The composition must automatically adapt to the discovered source footage.

For example:

durationInFrames = metadata-derived value

fps = metadata-derived value

width = metadata-derived value

height = metadata-derived value

Never assume 30 FPS, 1920x1080, 60 FPS, or 10 seconds unless the actual source metadata says so.

DEPENDENCY INSTALLATION

Install and integrate:

• remotion

• @remotion/transitions

• @remotion/motion-blur

• @remotion/effects

• @remotion/noise

• lucide-react

• @remotion/google-fonts

Also inspect whether additional dependencies are required for audio analysis, media metadata extraction, shader/canvas processing, color manipulation, and utility functions.

Only install additional packages when they provide a real benefit and are compatible with the current Remotion version.

Ensure all package versions are mutually compatible.

After installation:

1. verify package.json,

2. verify lockfile,

3. verify imports,

4. run TypeScript validation.

PHASE 1.5 — AUTOMATED MEDIA ANALYSIS

Before constructing the edit, analyze the source footage.

Visual characteristics:

• scene boundaries

• shot changes

• brightness

• contrast

• dominant colors

• motion intensity

• camera movement

• high-motion frames

• low-motion frames

• visually important segments

Audio characteristics:

• speech/dialogue regions

• music regions

• silence

• peaks

• transient/impact regions

• approximate loudness

• approximate RMS

• approximate dynamic range

Use this information to make the edit adaptive rather than applying identical effects across every frame.

If reliable automatic analysis is unavailable, implement a deterministic heuristic system rather than failing.

PHASE 2 — AE-GRADE COMPOSITING & EDIT AUTOMATION

Implement a modular, layered composition structure in src/MainComposition.tsx containing the following processing passes.

Create supporting components/utilities where appropriate.

Suggested architecture:

```
src/
```

■■■ MainComposition.tsx

■■■ Root.tsx

■■■ index.ts

■■■ components/

■ ■■■ VideoLayer.tsx

■ ■■■ CameraSystem.tsx

■ ■■■ MotionBlurLayer.tsx

■ ■■■ TransitionLayer.tsx

■ ■■■ Typography.tsx

■ ■■■ LowerThird.tsx

■ ■■■ ColorGrade.tsx

■ ■■■ FilmGrain.tsx

■ ■■■ ChromaticAberration.tsx

■ ■■■ Vignette.tsx

■ ■■■ AudioMix.tsx

■ ■■■ EffectsStack.tsx

■■■ utils/

■ ■■■ mediaMetadata.ts

■ ■■■ assetDiscovery.ts

■ ■■■ timing.ts

■ ■■■ easing.ts

■ ■■■ audioAnalysis.ts

■ ■■■ effects.ts

■■■ types/

■■■ media.ts

Adapt this structure to the existing project rather than blindly creating duplicates.

1. DYNAMIC CAMERA & DYNAMIC MOTION BLUR

Wrap transformed visual elements with from @remotion/motion-blur.

Use shutter angle: 180°.

Implement procedural handheld camera movement using:

• noise2D

• deterministic pseudo-random motion

• spring-based oscillation

• damped positional drift

The motion must remain subtle and cinematic.

Do not create nauseating or excessive movement.

BEAT / IMPACT-SYNCHRONIZED CAMERA SHAKE

Implement camera shake events triggered by:

• audio peaks

• detected impacts

• major transitions

• scene changes

• high-energy moments

Use decaying spring oscillations.

Shake intensity must decay naturally.

Avoid continuous random shaking.

DYNAMIC ZOOM PUNCHES / KEN BURNS

Implement dynamic scaling such as:

```
  scale: interpolate(
```

frame,

[0, duration],

[1.0, 1.08],

{

easing: Easing.bezier(...)

}

)

Make the behavior adaptive.

Support:

• slow cinematic push-in

• pull-out

• impact zoom

• focal-point zoom

• transition zoom

• subtle breathing motion

Avoid clipping important subjects.

2. NON-DESTRUCTIVE TIMELINE & ADVANCED TRANSITIONS

Structure timeline clips using .

Use intelligent cut points based on:

• source duration

• detected scene boundaries

• motion

• audio peaks

• pacing

• transition energy

Use:

• wipe()

• slide()

• fade()

and custom transition treatments:

• zoom transition

• directional blur

• light leak

• flash

• warp

• radial zoom

• motion smear

• RGB split

• exposure flash

Use:

springTiming({

config: {

damping: 15,

mass: 0.8

}

})

Transitions must be deterministic, render-safe, duration-aware, and visually coherent.

Never transition longer than the available clip duration.

3. KINETIC TYPOGRAPHY & MOTION GRAPHICS

Render typographic overlays using:

```
  spring({
```

frame,

fps,

config: {

damping: 12,

stiffness: 100

}

})

Implement:

• title cards

• subtitles where appropriate

• lower thirds

• section labels

• callouts

• metadata labels

• animated captions when source/dialogue data allows it

TYPOGRAPHY ANIMATION

Implement:

• word-staggered reveals

• letter-staggered reveals

• elastic bounce

• spring overshoot

• opacity interpolation
• vertical displacement
• scale interpolation
• tracking/letter-spacing animation
• masked reveals

Animation should be synchronized with the underlying footage.

GLASSMORPHISM

Use:

backdrop-filter: blur(16px);

with:

border: 1px solid rgba(255, 255, 255, 0.15);

and dynamic:

filter: drop-shadow(...);

Add:
• subtle gradients
• translucent panels
• animated highlights
• soft edge lighting
• restrained glow

Keep the design cinematic rather than overly UI-like.

4. COLOR GRADING, LIGHTING & OPTICAL FX

Grade the footage layer through Remotion-compatible CSS/canvas/effect passes.

Implement:
• contrast
• saturation
• exposure
• brightness
• highlights
• shadows
• color temperature where feasible
• cinematic tonal curve
• soft vignette

Create an effects stack where effects can be independently enabled/disabled.

HIGH-IMPACT OPTICAL EFFECTS

At selected high-impact frames and transitions, implement subtle:

• chromatic aberration
• RGB split
• film grain
• exposure flash
• bloom/glow
• directional blur
• lens distortion
• light leak
• motion smear

Effects must be subtle by default.

Do not apply every effect to every frame.

5. FILM GRAIN & PROCEDURAL TEXTURE

Implement deterministic procedural film grain.

Grain intensity should vary based on:
• scene energy
• transitions
• darkness
• high-impact moments

Do not use uncontrolled randomness that causes frame-to-frame render inconsistency.

Ensure procedural effects are deterministic for identical render inputs.

6. AUDIO MASTERING & AUTO-DUCKING PIPELINE

Ingest original and background audio via or depending on actual Remotion compatibility.

Support:
• original source audio
• background music
• voice/dialogue
• ambience
• sound effects

when available.

AUDIO ENVELOPE

Start:

Smooth exponential attack fade-in: 0.5 seconds

End:

Smooth release fade-out: 1.0 second

Do not create audible clicks or abrupt volume changes.

AUTOMATIC DIALOGUE DUCKING

Automatically detect or use known vocal/dialogue regions.

Whenever vocal/dialogue is active:

duck background music by 65%.

The ducking transition must itself be smoothed.

Avoid hard volume jumps.

If dialogue detection is unavailable, implement a configurable timing/region system that can be automatically generated from available audio analysis.

7. AUDIO PEAK / BEAT SYNCHRONIZATION

Use detected audio transients and peaks to drive:

• camera shake

• zoom punches

• typography entrances

• transition timing

• light flashes

• RGB split

• grain intensity

• motion blur intensity

Create a reusable event model such as:

AudioEvent {

frame

intensity

type

}

Possible event types:

• beat

• impact

• speech_start

• speech_end

• silence

• transition

All generated events must remain deterministic.

8. ADAPTIVE CINEMATIC PACING

Do not simply divide the video into equal-length sections.

Generate pacing based on:

• scene changes

• motion

• audio energy
• speech
• silence
• visual intensity
• source duration

Low-energy footage should receive slower transitions.

High-energy footage may receive:
• faster cuts
• punch zooms
• stronger motion blur
• stronger transition effects

but remain aesthetically controlled.

9. RESPONSIVE / SOURCE-AWARE COMPOSITION

The composition must automatically adapt to:
• landscape footage
• portrait footage
• square footage
• unusual resolutions

Do not stretch the video unnaturally.

Use appropriate:
• contain
• cover
• crop
• focal-point positioning

logic.

Preserve the source aspect ratio.

10. SAFE-AREA & SUBJECT PROTECTION

When adding typography:
• avoid important faces
• avoid obvious subjects
• avoid cutting off the main action
• avoid existing subtitles
• avoid UI elements in the source

If automated subject detection is unavailable, use configurable safe regions and source-aware placement heuristics.

11. LAYER ORDER

Maintain a deliberate compositing order similar to:

Background

↓

Source Video

↓

Color Grade

↓

Lighting

↓

Motion Blur

↓

Optical Effects

↓

Film Grain

↓

Vignette

↓

Transitions

↓

Typography

↓

Lower Thirds

↓

UI / Icons

↓

Final Exposure / Composite Pass

Adjust this ordering where technically necessary.

12. PERFORMANCE & RENDER OPTIMIZATION

The final composition must remain renderable.

Avoid:

• unnecessary per-frame heavy computation

• huge unbounded arrays

• uncontrolled DOM growth

• repeated expensive calculations

• nondeterministic effects

• browser APIs unavailable to Remotion

• unnecessary canvas recreation

Use memoization/caching where appropriate.

Precompute expensive deterministic data when possible.

Keep frame calculations efficient.

13. RENDER-SAFE DETERMINISM

Every render of the same:

- source footage
- assets
- code
- configuration

must produce the same animation timing.

Do not use Math.random() directly inside frame rendering unless seeded deterministically.

Use deterministic noise.

14. CONFIGURATION SYSTEM

Create a centralized configuration system so the cinematic pipeline can be tuned without rewriting components.

Example categories:

- camera
- motionBlur
- transitions
- typography
- colorGrade
- grain
- chromaticAberration
- audio
- ducking
- timing
- performance

Allow settings such as:

- cameraIntensity
- zoomIntensity
- shakeIntensity
- grainIntensity
- vignetteIntensity
- transitionDuration
- musicDuckAmount
- dialogueDuckAmount

The required default music ducking amount is 65%.

15. ERROR HANDLING & FALLBACKS

Implement graceful fallbacks for:

- missing audio
- missing music
- missing fonts
- missing graphics
- unsupported asset formats

• metadata extraction failure
• absent dialogue
• absent scene detection
• unavailable optional effects

The render should not crash simply because an optional asset is missing.

Provide sensible defaults.

16. FONT & ICON SYSTEM

Integrate:
• @remotion/google-fonts
• lucide-react

Use modern cinematic typography.

Choose fonts that are available deterministically during rendering.

Do not make the render depend on an unreliable runtime network request.

If necessary, configure/fallback to a locally available system-safe font.

Use Lucide icons for subtle:
• arrows
• indicators
• metadata
• labels
• UI accents

Do not overload the composition with icons.

17. REMOTION ROOT CONFIGURATION

Ensure src/Root.tsx correctly registers:
• the main composition
• dynamic width
• dynamic height
• dynamic FPS
• dynamic duration

based on the discovered media metadata.

Do not hardcode these values unless used as fallback defaults.

Validate that the composition ID is exactly:

MainComposition

18. ENTRY POINT

Ensure src/index.ts is a valid Remotion entry point.

Verify:

• imports

• composition registration

• bundling

• rendering

• TypeScript compatibility

PHASE 3 — VALIDATION, ERROR RECOVERY & AUTONOMOUS RENDER EXECUTION

TYPESCRIPT AUDIT

Compile and audit TypeScript types.

Ensure:

• zero TypeScript errors

• no unresolved imports

• no invalid Remotion APIs

• no incompatible component props

• no missing modules

• no invalid CSS object values

• no runtime-only browser assumptions

Specifically validate:

```
src/Root.tsx
src/index.ts
```

and every newly created component.

ASSET RESOLUTION VALIDATION

Verify all assets resolve using staticFile() or another Remotion-safe mechanism.

There must be:

• no hardcoded local absolute paths

• no machine-specific paths

• no broken asset references

• no references to nonexistent files

REMOTION BUNDLE VALIDATION

Run the appropriate Remotion bundle/build validation.

If bundling fails:

1. inspect the exact error,

2. identify the failing module,

3. correct the implementation,

4. rerun the bundle,

5. continue until successful.

FRAME-LEVEL VALIDATION

Before the final production render, validate representative frames such as:

• first frame

• early frame

• transition frame

• high-motion frame

• typography frame

• middle frame

• audio-impact frame

• final frame

Check for:

• clipping

• black frames

• missing assets

• broken effects

• typography overflow

• transition artifacts

• excessive camera movement

• incorrect scaling

• unexpected transparency

• render crashes

FINAL RENDER

Automatically execute:

```
npx remotion render src/index.ts MainComposition out/final_render.mp4 --concurrency=8 --gl=angle --crf=18
```

If the exact Remotion CLI syntax differs because of the installed version, adapt the command to the installed version while preserving the requested render settings as closely as possible.

Required target:

```
out/final_render.mp4
```

RENDER VERIFICATION

After rendering, verify:

• file exists

• file is non-zero size

• video duration is correct

• FPS is correct

• resolution is correct

• audio exists when expected

• final frame is present

• output is playable

• no fatal FFmpeg errors occurred

Use FFprobe or another reliable metadata mechanism to verify the final file.

FINAL QUALITY CONTROL

VIDEO

• correct duration

• correct FPS

• correct resolution

• correct aspect ratio

• no black frames

• no missing footage

• no broken transitions

• no excessive effects

• no frozen frames

MOTION

• smooth camera movement

• natural motion blur

• controlled shake

• coherent zooms

• smooth transition timing

TYPOGRAPHY

• readable

• correctly positioned

• no clipping

• correct animation

• safe margins

COLOR

• consistent grade

• controlled contrast

• no crushed blacks

• no blown highlights

• restrained optical effects

AUDIO

• no clipping

• correct fade-in

• correct fade-out

• dialogue remains intelligible

• background music ducks by 65% during dialogue

• no obvious clicks

AUTONOMOUS ERROR-RECOVERY LOOP

If any stage fails, do NOT stop immediately.

Use:

INSPECT
↓
DIAGNOSE
↓
PATCH
↓
TYPECHECK
↓
BUNDLE
↓
TEST
↓
RENDER
↓
VERIFY

Repeat until successful or until a genuinely external limitation makes completion impossible.

Do not hide errors.

Do not claim a render succeeded unless the output file was actually generated and verified.

IMPORTANT IMPLEMENTATION PRINCIPLES

DO NOT FAKE FUNCTIONALITY

If a requested effect cannot be implemented using a particular Remotion package/API, do not create a fake import.

Instead:

1. inspect the installed package/API,
2. determine the correct implementation,
3. use the closest technically correct Remotion-compatible implementation.

API VERSION AWARENESS

Before using APIs such as:

- CameraMotionBlur
- TransitionSeries
- springTiming
- wipe
- slide
- fade
- noise2D

• spring

• Remotion effects

verify that the installed package version actually exposes the API.

If the API signature differs, adapt the implementation to the installed version.

PROJECT OUTPUT REQUIREMENTS

The completed project must contain a working automated pipeline capable of:

Raw Assets

↓

Asset Discovery

↓

Metadata Extraction

↓

Media Analysis

↓

Timeline Generation

↓

Camera System

↓

Motion Blur

↓

Transitions

↓

Kinetic Typography

↓

Color Grade

↓

Optical FX

↓

Film Grain

↓

Audio Mixing

↓

Dialogue Ducking

↓

Final Composite

↓

Remotion Bundle

↓

FFmpeg Render

↓

Quality Verification

↓

```
out/final_render.mp4
```

IMPORTANT: DO NOT STOP AT CODE GENERATION

Your task is not complete when the source code has been written.

Your task is complete only when:

• dependencies are installed,

• project architecture is implemented,

• assets are detected,

• metadata is detected,

• composition is registered,

• TypeScript passes,

• Remotion bundles successfully,

• render executes successfully,

• out/final_render.mp4 exists,

• final output metadata is verified,

• and no known blocking runtime errors remain.

At the end, provide a concise execution report containing:

1. detected source asset
2. source resolution
3. source FPS
4. source duration
5. source frame count
6. detected companion assets
7. installed/used Remotion packages
8. implemented processing passes
9. validation status
10. final render path
11. final render resolution/FPS/duration
12. any non-blocking limitations

Do not provide a hypothetical report. Populate it from the actual project execution results.